

Instituto Tecnológico de Costa Rica

Escuela Ingeniería en Computadores

Curso: CE1101 Introducción a la programación

Documentación técnica

OvercookTEC

Estudiantes:

Ignacio José Apuy Anchía

Jordanny Steven Hernández Mora

Profesor:

Leonardo Araya Martínez

Semestre I 2026

0. Índice

0. Índice	2
1. Introducción	3
1.1. Propósito del documento	3
1.2. Descripción del proyecto	3
1.3. Objetivos y Alcance	4
1.4. Definiciones, Acrónimos y Abreviaturas	4
2. Descripción General del Sistema	4
1.1. Perspectiva del Producto	5
1.2. Funcionalidades principales	5
1.3. Restricciones del sistema, Supuestos y Dependencias	5
3. Requisitos del Sistema	6
2.1. Requisitos Funcionales	6
2.2. Requisitos No Funcionales	6
4. Diseño de la Solución	8
3.1. Visión General de la Arquitectura	8
3.2. Descripción de Componentes	9
3.3. Stack Tecnológico	10
3.4. Diagrama de Clases y Modelo de Datos	10
3.5. Diagrama de Flujo y Secuencia	12
3.6. Decisiones de Diseño y Justificación	14
5. Implementación y Validación de Calidad	15
4.1. Estructura del repositorio	15
4.2. Entorno de Desarrollo, Guía de instalación y Ejecución	15
4.3. Estrategias y Casos de Prueba	16
4.4. Resultados de pruebas y problemas encontrados	19
6. Declaración de uso de Inteligencia Artificial y herramientas	22
5.1. Herramientas de IA utilizadas	22
5.2. Descripción del Uso de Inteligencia Artificial	22
5.3. Declaración de Responsabilidad y Autoría	23
7. Conclusión	24
8. Bibliografía	25

1. Introducción

OvercookTEC es un proyecto enfocado en la utilización de lenguaje de programación Python, y centrado en utilizar la programación orientada a objetos (POO). A través de esto se replica, en un ambiente de aplicación de software, las funcionalidades, comportamientos, estilos y la idea general del videojuego Overcooked 2, de Ghost Town Games y Team17.

Mediante análisis y utilizando conocimiento previo en programación, se aplica una hipótesis del funcionamiento de las diversas mecánicas que existen dentro del videojuego original, y cómo esto se puede plasmar utilizando programación orientada a objetos.

A continuación, se destacarán diferentes aspectos y logísticas que se tomaron en cuenta para la realización de este proyecto, y, por lo tanto, en su desarrollo.

1.1. Propósito del documento

El presente documento provee una descripción técnica y completa de los métodos y estrategias utilizados para el desarrollo de la aplicación de software, así como características e información adicional del programa, y se tiene como destino a investigadores o evaluadores que requieran de esta información para diversos análisis.

1.2. Descripción del proyecto

El sistema en el que se basa el proyecto forma parte del ámbito de aplicaciones de escritorio para computadoras. Este se crea a partir de una biblioteca del lenguaje de programación Python, llamada Tkinter, la cual permite crear ventanas que permiten a los usuario interactuar con el programa, y de igual manera manejar las entradas por código.

Como público objetivo se tienen a usuarios, aficionados a videojuegos, que junto a otra persona deseen poner a prueba sus habilidades de coordinación para cumplir el objetivo. Así también se tienen a investigadores y/o programadores con interés en conocer el funcionamiento del videojuego.

1.3. Objetivos y Alcance

- Objetivo general: Desarrollar una aplicación de escritorio interactiva, para dos jugadores utilizando el lenguaje Python enfocado a programación orientada a objetos, y la biblioteca Tkinter para crear una réplica del videojuego Overcooked 2.
- Objetivos específicos:
 - Desarrollar un videojuego para dos personas en el que ambos puedan interactuar con la aplicación, de manera local y simultáneamente, y que a su vez, el sistema pueda manejarlo.
 - Mantener un enfoque orientado a objetos que gestione las diversas ventanas, los escenarios, objetos, recetas y las propiedades de las estaciones
 - Crear un sistema de interfaz gráfica (GUI), agradable e intuitivo, para ambos usuarios. que permita interactuar con la aplicación y movilizarse a través de esta.
- Alcance: Mediante este proyecto se busca crear una réplica básica del videojuego original. El sistema se limitará a tres escenarios básicos donde solo dos jugadores pueden interactuar en ellos. Se incluyen únicamente mecánicas básicas, como las estaciones de cocina, picado y entrega.

1.4. Definiciones, Acrónimos y Abreviaturas

- Aplicación de escritorio: Estas se componen de ventanas y elementos, y conforman entornos donde se puede interactuar, ingresar datos, procesarlos, generar respuestas y demás, con el fin de resolver problemas o realizar acciones específicas en un sistema operativo.
- GUI: Una interfaz gráfica o Graphic User Interface por sus siglas en inglés son diseños creados para que un usuario pueda visualizar e interactuar con un programa mediante elementos.
- Python: Lenguaje de programación popular y altamente utilizado por su versatilidad y eficacia para resolver problemas y su aplicabilidad a tareas cotidianas.
- Biblioteca (De un lenguaje de programación): Las bibliotecas son funciones y/o métodos predefinidos de programación creados por terceros para ser utilizados en programas con mayor facilidad y accesibilidad.
- POO: La Programación Orientada a Objetos es una metodología utilizada en programación, donde se crean clases u objetos que contienen características propias; pueden ser instanciados o llamados, se pueden heredar o pasar a otras partes del programa, y entre otras funcionalidades. Se destina esto a crear flujos profesionales dentro de la ejecución del programa.
- Bug: Este término se aplica a errores dentro del funcionamiento del código de un programa que suelen ser pequeños pero difíciles de encontrar o solucionar.

2. Descripción General del Sistema

En esta sección se proveen descripciones y factores generales sobre el concepto, desarrollo y producto resultante de este proyecto, así como detalles de su implementación y funcionamiento.

1.1. Perspectiva del Producto

El programa se opera en el sistema operativo Windows 11, junto al lenguaje Python, encargado de revisar y ejecutar el código fuente. El programa funciona localmente, dentro de su carpeta de proyecto y entre sus archivos fuente, y no requiere de otros sistemas externos.

1.2. Funcionalidades principales

- Manejo de secciones: Desplazamiento entre ventanas.
- Creación de escenarios: Escenarios únicos con propiedades distintas.
- Gestión de partidas: Eventos, puntuación, veredictos.
- Movimiento del jugador: Movimientos e interacciones con objetos en la escena.
- Acciones: Procesos entre el jugador, los objetos y las estaciones.

1.3. Restricciones del sistema, Supuestos y Dependencias

- Restricciones:
 - El sistema debe ejecutarse en un sistema operativo Windows 11 de 64 bits, ya que no se realizaron pruebas en otros sistemas.
 - La resolución mínima del monitor no debe ser menor a 1366x768.
- Supuestos:
 - El usuario debe tener instalado Python 3.x antes de ejecutar el programa.
- Dependencias:
 - Librerías de Python (Ya vienen incluidas por defecto).

3. Requisitos del Sistema

Se destacan diferentes requisitos funcionales y no funcionales del sistema. Ambos son métricas fundamentales para definir adecuadamente el comportamiento correcto del programa y generar un grado de satisfacción en el usuario.

2.1. Requisitos Funcionales

ID	Descripción	Prioridad	Criterio de Aceptación
RF-001	Sistema de escenarios	Alta	Los escenarios pueden seleccionarse a elección del usuario. Se dividen por casillas, y cada personaje debe interactuar con una casilla según su propósito
RF-002	Movilidad de personajes	Alta	Los personajes se mueven libremente por el escenario. Colisionan e interactúan con las casillas requeridas
RF-003	Sistema de ingredientes	Alta	Cada ingrediente tiene su propio método de preparación según valores predefinidos
RF-004	Sistema de pedidos	Alta	Se generan pedidos aleatorios, predefinidos según el escenario en el que se juega
RF-005	Manejo de tiempos	Alta	Cada partida debe tener una duración predeterminada. Así también para los tiempos de cocción y demás mecánicas que lo requieran

2.2. Requisitos No Funcionales

ID	Categoría	Descripción	Métrica / Valor Esperado
RNF-001	Rendimiento	El sistema responde a las entradas dadas por ambos usuarios	Se espera que el sistema responda inmediatamente o en un lapso casi inmediato para no degradar el rendimiento general.
RNF-002	Diseño de interfaz	La interfaz debe estar organizada e intuitiva para el usuario final.	Interfaz legible, clara, concisa, ordenada y capaz de guiar al usuario de forma autónoma.

RNF-003	Usabilidad	Debe haber respuesta de la interfaz hacia el usuario, y ser visualmente atractiva	interacciones eficaces e intuitivas para no confundir o angustiar al usuario.
---------	------------	---	---

4. Diseño de la Solución

En esta sección se describen las decisiones y modelos de arquitectura, diseño y datos, los componentes, y los diagramas que se utilizaron para y durante el desarrollo del proyecto.

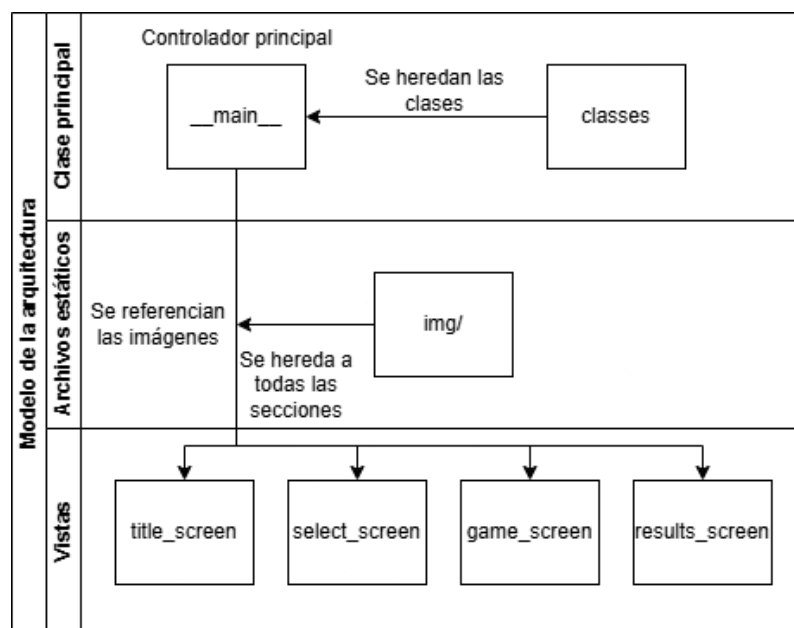
3.1. Visión General de la Arquitectura

Este proyecto utiliza una arquitectura basada en un controlador central. El archivo principal (`__main__.py`) actúa como un moderador, y es el encargado de traer al frente a las diferentes vistas (screens) y proveerles métodos para un funcionamiento aislado pero interconectado por el controlador y sus propiedades heredadas.

Se decidió utilizar este diseño debido a su fácil interoperabilidad entre programas aislados, además de que la librería Tkinter se aprovecha mejor en casos aislados con sus propiedades específicas.

A continuación, se presenta un diagrama descrito, que representa esta arquitectura y su funcionamiento. Las capas se describen de la siguiente forma:

- Capa de presentación (vistas): Aquí se encuentran las diferentes vistas o ventanas por clase que se intercambian para mostrar un contenido específico para una ocasión dada. Estos son por ejemplo `title_screen`, `select_screen`, `game_screen`, etc., que conforman distintas secciones dentro del flujo del programa.
- Capa de lógica: Aquí se presentan como modelos las diferentes clases compartidas entre vistas y el controlador. Estos se definen en el archivos `classes.py`, donde se encuentran `Player` y `Escenario` como modelos importantes.
- Capa de datos: Esta se compone de archivos estáticos, que corresponden a imágenes en `img/`.



3.2. Descripción de Componentes

Componente	Responsabilidad	Tecnología	Dependencias
Frontend	Interfaz de usuario y experiencia visual.	Tkinter	Librería Tkinter (Incluida en Python)
Backend	Lógica y funcionamiento interno de la aplicación	Python	Librería random (Incluidas en Python)

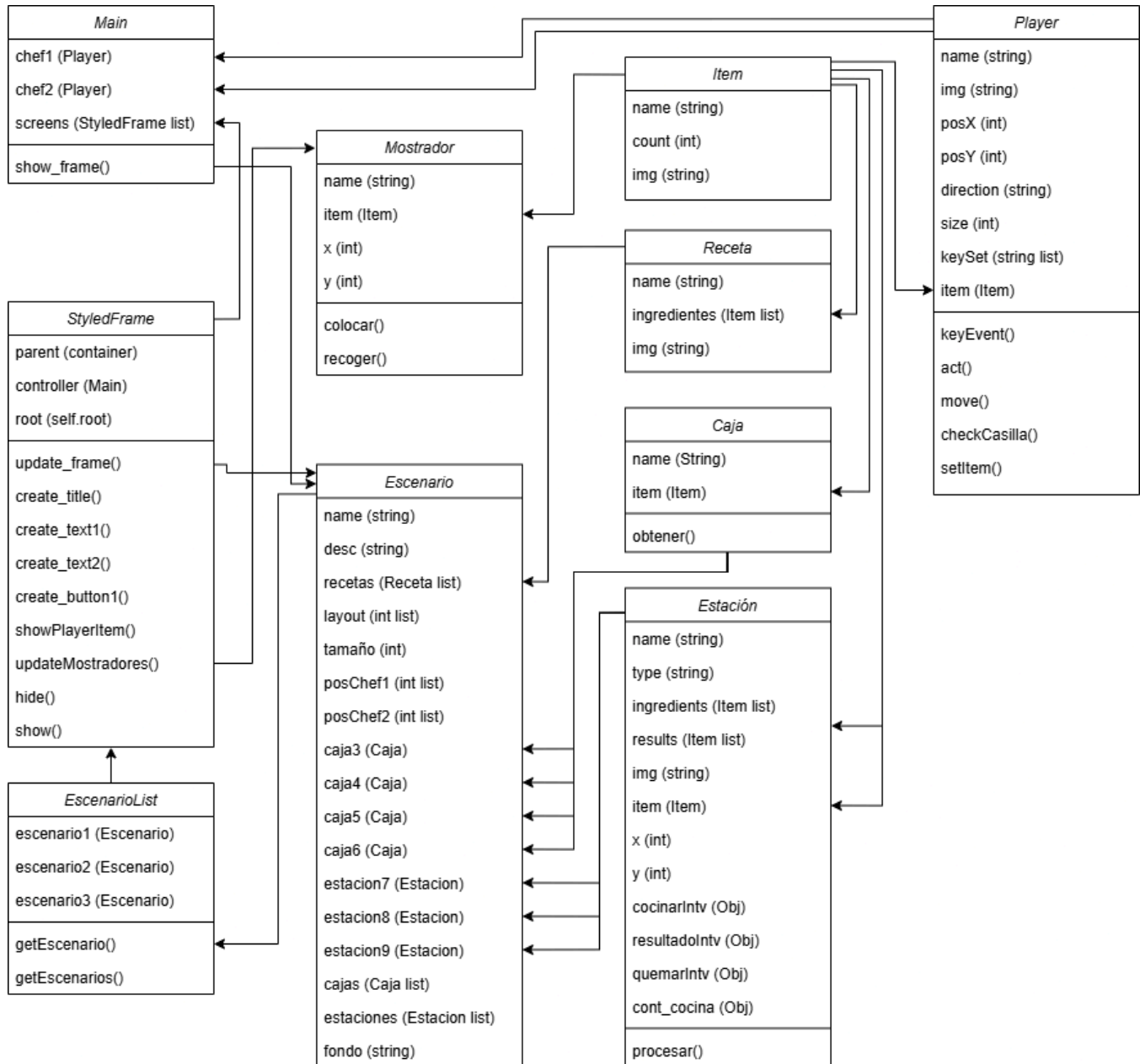
3.3. Stack Tecnológico

Categoría	Tecnología	Versión	Propósito
Framework (Frontend)	Tkinter	8.6.15	Creación de GUI
Lenguaje (Backend)	Python	3.14.3	Lógica y funcionamiento interno del programa

3.4. Diagrama de Clases y Modelo de Datos

El siguiente es un diagrama de cómo interactúan las clases del programa entre sí, así como sus distintas propiedades, métodos y subclases si las poseen. Las clases se describen a continuación como:

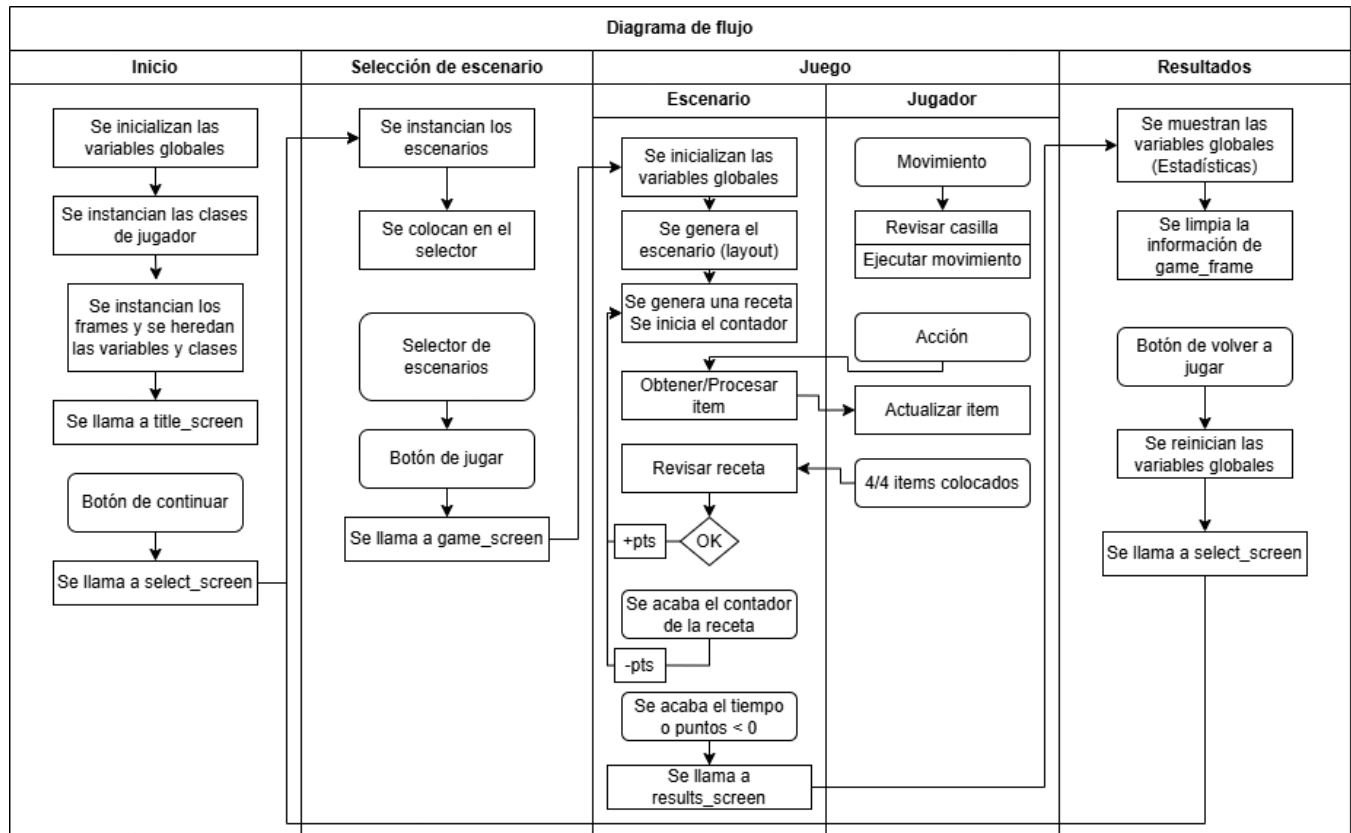
- Clase Main: Esta es la clase encargada de iniciar todo el programa, controlar las ventanas, heredar los modelos de los jugadores, así como variables globales como la puntuación y los pedidos para los resultados..
- Clase StyledFrame: Esta clase se hereda a cada ventana por medio de Main como controlador. Esta clase crea la interfaz de la sección que se desea mostrar, y posee métodos para crear textos y botones, ocultar y mostrar elementos, registrar eventos, entre otras propiedades.
- Clase Player: Contiene el nombre, imagen, configuración de teclas, así como sus posición y dirección. Se instancia como chef sobre el escenario. Contiene métodos para ejecutar movimientos y acciones sobre otros objetos dentro del escenario.
- Clase Escenario: Esta clase contiene información sobre cómo se crearán las paredes, suelos, estaciones y demás objetos a modo de entorno. Contiene gran cantidad de atributos como recetas posibles, funciones de las cajas y estaciones, imágenes, posiciones absolutas y demás.
- Clase EscenarioList: Es una clase de clases Escenario, donde se guardan y referencian algunos escenarios ya predefinidos para seleccionarlos en el juego.
- Clase Item: Esta clase simple contiene el nombre, cantidad y textura de un objeto.
- Clase Caja: Contiene dentro un Item específico que se puede obtener por medio de un método propio.
- Clase Estacion: Se divide en diferentes tipos según los objetos que procesa y cómo lo hace. Dentro se encuentran ingredientes que puede admitir y resultados de ellos. También se incorporan intervalos para cuando se cocina, ya que los Item se ven afectados según cuanto tiempo se cocina.
- Clase Mostrador: Es otro tipo de Estación que puede guardar Items temporalmente o bien comparar los Items guardados con las Recetas para puntuar.
- Clase Receta: Contiene diferentes Items que componen una receta posible en específico.



3.5. Diagrama de Flujo y Secuencia

A continuación, se presenta un diagrama de flujo para el funcionamiento básico y esperado del programa una vez en ejecución.

- Código principal: Al iniciar el programa, se inicializan las variables globales, se instancian las clases, se heredan los atributos y se trae al frente la primera ventana. Esta primera ventana contiene instrucciones e imágenes del juego, junto a un botón para continuar. Además, en todo momento, en una barra superior, existe un botón para salir del programa.
- Selección de escenario: Llama a los escenarios en EscenarioList y los presenta como diferentes opciones, donde se muestran sus nombres e imágenes y se le pide al jugador escoger uno de ellos para continuar.
- Sección del juego: Se divide en Escenario y Jugador, ya que interactúan entre sí sin ser el mismo concepto. El escenario se encarga de crear todos los bloques (tiles) por donde puede o no pasar el jugador, o bien interactuar con ellos. Después se genera la primera receta, se colocan los contadores. Finalmente se procesan las diferentes acciones del jugador. Por otro lado, el jugador puede moverse libremente por los bloques vacíos, obtener Items de las Cajas, procesarlos en las Estaciones, y colocarlos en los Mostradores. En general, si el tiempo de la partida se acaba o el puntaje llega a cero, se termina el juego.
- Resultados: Una vez que el juego termina, se limpia el escenario cargado previamente y se muestran los resultados de la partida que se guardaron en las variables globales. Por último se muestra un botón para volver a jugar otra partida.



3.6. Decisiones de Diseño y Justificación

Decisión	Alternativas Evaluadas	Opción Elegida	Justificación
Lenguaje	Python C# Java	Python	Se estableció en las indicaciones del proyecto utilizar este lenguaje imperativamente.
Arquitectura	Modelo Vista-Controlador Aplicación simple Modelo de controlador único	Modelo de controlador único	Este patrón de modelo es suficiente para lograr el concepto del proyecto, además de que es adecuado para utilizar modelos de forma global y constante a través de toda la ejecución.
Framework	Tkinter CustomTkinter Pygame	Tkinter	Se cuenta con más experiencia en la utilización de Tkinter que con las demás librerías evaluadas
Editor de código (IDE)	IntelliJ Visual Studio Code Visual Studio	Visual Studio Code	Este IDE es bastante flexible y eficaz para poder darle vida a cualquier proyecto. Se optó por su facilidad, integración y rendimiento.

5. Implementación y Validación de Calidad

En esta sección se describe la estructura del repositorio donde reside el código, el entorno y herramientas utilizadas para el desarrollo del proyecto, los pasos para ejecutarlo, las pruebas realizadas y los resultados de las mismas.

4.1. Estructura del repositorio

Ruta	Archivo	Descripción
Proyecto_OvercookTEC/	__main__.py	Archivo con la clase principal. Motor del proyecto.
Proyecto_OvercookTEC/	Run.bat	Archivo ejecutable para iniciar el juego rápidamente.
Proyecto_OvercookTEC/	Documentación Externa OvercookTEC.docx	Documentación externa del proyecto (Este documento)
Proyecto_OvercookTEC/	Documentación Atributos OvercookTEC.docx	Documentación de atributos del proyecto
Proyecto_OvercookTEC/assets/img/	[Varios archivos de imágenes]	Imágenes de los platos, los bloques, logos y capturas de pantalla.
Proyecto_OvercookTEC/assets/	classes.py	Archivo con las definiciones de clases (Player, Escenario, Caja, Mostrador...) y sus métodos.
Proyecto_OvercookTEC/assets/	game_screen.py	Archivo con la sección donde se muestra el escenario y se ejecutan los pedidos y mecánicas.
Proyecto_OvercookTEC/assets/	results_screen.py	Archivo con la sección de resultados de la partida.
Proyecto_OvercookTEC/assets/	select_screen.py	Archivo con el menú para seleccionar el escenario.
Proyecto_OvercookTEC/assets/	title_screen.py	Archivo con la sección de menú principal.

4.2. Entorno de Desarrollo, Guía de instalación y Ejecución

- Entorno:
 - Sistema Operativo: Windows 11
 - Lenguaje de programación: Python 3.13.x
 - Se requiere Git para clonar el repositorio
- Guía de instalación y ejecución:
 - 1. Abrir la terminal de Windows y clonar el repositorio con:
git clone https://github.com/NaxoW77/Proyecto_OvercookTEC.git
 - 2. Una vez clonado, abrir el archivo ejecutable Run.bat

4.3. Estrategias y Casos de Prueba

- Estrategias de pruebas:

Tipo de Prueba	Descripción	Herramienta	Cobertura Objetivo
Pruebas funcionales aisladas	Se modificó el código para ejecutar secciones únicas, para realizar pruebas aisladas a diferentes partes del programa y su flujo propio.	Manualmente	Se busca probar cada ventana por separado y prepararlas para las pruebas totales.
Pruebas funcionales totales	Se realizaron pruebas de inicio a fin de cada flujo del programa, con la intención de encontrar problemas durante una ejecución normal, como usuario final y con cierta exigencia.	Manualmente	Casos generales, cubriendo una gran mayoría, por no decir la totalidad del programa.
Pruebas minuciosas o específicas	Se buscan comportamientos inusuales en casos específicos o difíciles de replicar, en puntos extremos del funcionamiento del programa.	Manualmente	Casos especiales o poco frecuentes donde se ingresan valores extremos y se buscan errores específicos, difíciles de provocar, pero posibles.
Pruebas de usuario final	Se tiene como objetivo que un usuario final externo al desarrollo de la aplicación interactúe con esta misma	Manualmente	Se pretende que la aplicación funcione correctamente cuando es utilizada por un

- Casos de prueba:

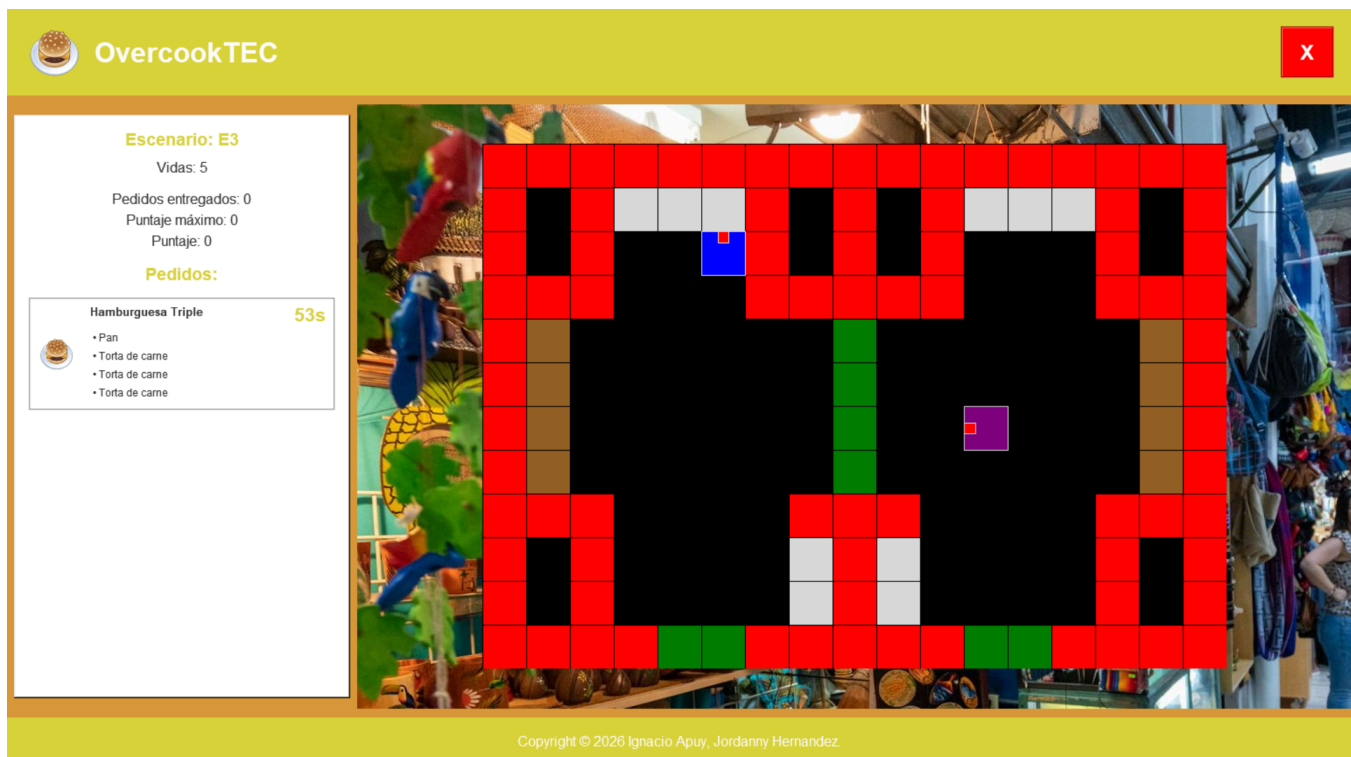
ID	Descripción	Pasos	Resultado Esperado	Resultado Obtenido	Estado
----	-------------	-------	--------------------	--------------------	--------

CP-001	Ejecución de la aplicación	1. Ejecutar el archivo run.bat	La aplicación inicia correctamente, y obtiene datos requeridos de los assets	Se ejecuta la aplicación correctamente	✓ Pasó
CP-002	Cálculo del puntaje según el pedido completado	1. Ingresar al juego. 2. Seleccionar un escenario 3. Completar pedidos.	El puntaje al entregar el pedido se calcula en base al tiempo restante y se muestra en pantalla	Se calcula correctamente el puntaje. Sin embargo en algunos casos se muestra el puntaje seguido de varios decimales	✓ Pasó
CP-003	Movimiento e interacción de los jugadores	1. Ingresar al juego. 2. Seleccionar un escenario 3. Mover personajes	El movimiento es acorde a las teclas definidas, y se puede interactuar con el escenario	Ambos personajes responden a su entrada respectiva. Sin embargo, al coincidir diferentes teclas, es posible que no todas respondan.	✓ Pasó
CP-003	Generación de recetas aleatorias	1. Ingresar al juego. 2. Seleccionar un escenario. 3. Completar pedidos	Los pedidos se generan aleatoriamente, y aumentan su dificultad entre más pedidos completados existan	Se generan pedidos predefinidos por escenario. Entre más pedidos completados existen, es posible que aumenten en cantidad los nuevos pedidos	✓ Pasó
CP-004	Registro de ítems por personaje	1. Ingresar al juego. 2. Seleccionar un escenario	Cada personaje por separado obtiene ítems propios de una	Cuando un personaje interactúa con una caja,	✓ Pasó

		3. Obtener ítems de las cajas	caja. Cada ítem tiene su propio procesamiento.	recibe el ítem respectivo y cumple con su propio proceso de preparación.	
CP-005	Finalización y reinicio de rondas	1. Ingresar al juego. 2. Seleccionar un escenario. 3. Completar una partida 4. Iniciar otra partida nueva	Los valores de la última ronda no deberían interferir en la siguiente ronda.	Las partidas se reinician correctamente, ningún dato previo interfiere con la nueva partida.	✓ Pasó
CP-006	Cerrar el programa durante una partida.	1. Ingresar al juego. 2. Seleccionar un escenario. 3. Completar una partida.	No debería ocurrir ningún problema. Los datos en uso deberían descartarse	Los datos en uso durante la partida se descartan correctamente	✓ Pasó
CP-007	Cambiar la resolución del monitor, mientras se ejecuta el juego.	1. Ejecutar el juego 2. Cambiar la resolución del monitor en las configuraciones del sistema.	Los elementos se deberían de mantener en orden.	Los elementos se quedaron en sus posiciones, aunque se aumentó su tamaño. Pero era esperado.	✓ Pasó

4.4. Resultados de pruebas y problemas encontrados

- Resultados:
 - Casos de prueba exitosos:
 - Las pruebas aisladas se realizaron con éxito. Se utilizó un “modo debug” para ocultar las texturas y ver el funcionamiento (Primera imagen demostrativa). Se encontraron errores mínimos los cuales fueron resueltos inmediatamente.
 - Las pruebas totales se realizaron y se tomaron gran cantidad de casos posibles, por lo que se extendió su duración. Un gran porcentaje de las pruebas fueron exitosas, aunque los principales errores provenían del canvas (texturas base) y los Item moviéndose a través de las estaciones y el jugador. Todos estos errores fueron corregidos oportunamente.
 - También se realizaron las pruebas extremas, donde se modificó el programa enormemente y se forzaron interacciones. Se encontraron ciertos errores que fueron corregidos de manera cuidadosa.
 - Imágenes demostrativas y de prueba





OvercookTEC



Escenario: E2

Vidas: 5

Pedidos entregados: 0

Puntaje máximo: 0

Puntaje: 0

Pedidos:

"Uno con todo"

58s



- Lechuga picada
- Tomate picado
- Tronaditas
- Fresco



Copyright © 2026 Ignacio Apuy, Jordanny Hernandez.



OvercookTEC



Resultados

Resultados de la partida:

Puntaje: 80.0

Pedidos entregados: 12

Volver a jugar

Copyright © 2026 Ignacio Apuy, Jordanny Hernandez.

- Problemas Solucionados: .
 - Al jugar una nueva partida tras otra, los escenarios colisionaban en su creación.
 - Las interacciones del jugador no se ejecutaban correctamente en el plano del escenario.
 - Los items de los jugadores, las estaciones y los mostradores no se mostraban.
 - Los items en los mostradores no se comparan adecuadamente con las recetas contenidas en los pedidos.
 - Los contadores para las estaciones de cocina se cruzaban y proveían resultados incorrectos.
- Problemas no solucionados:
 - Se encontraron soluciones a los problemas mencionados, además de otros menos importantes. Fuera de las limitaciones mencionadas anteriormente por entradas de teclado, no se encontraron problemas que puedan llegar a ocurrir ni que afecten la experiencia del juego.

6. Declaración de uso de Inteligencia Artificial y herramientas

5.1. Herramientas de IA utilizadas

Herramienta de IA	Tipo	Período de Uso	Propósito
Gemini	Modelo de lenguaje y generación de texto	Casi nunca.	Generación de redacción redundante o poco importante.
Copilot	Asistente de código	Rara vez.	Corrección de errores mínimos de sintaxis y otros.
Windsurf	Asistente de código	Usual.	Autocompletado de código.

5.2. Descripción del Uso de Inteligencia Artificial

- Gemini: Se utilizó este modelo de lenguaje para generar textos y corregir ideas que en conjunto no significan una parte importante de todo el proceso como tal. Cabe destacar que se utilizó en ocasiones muy específicas, por lo que no se usó frecuentemente.
- Copilot: Se utilizó este asistente en pocas ocasiones para corregir problemas de sintaxis y funcionamientos inusuales (bugs). Este uso se dio a raíz de errores pequeños y difíciles de encontrar, donde se necesitó utilizar la herramienta para agilizar el proceso de buscarlos y corregirlos rápidamente. En cada ocasión se compararon las líneas previas con las nuevas generadas por el asistente para verificar su validez y concepto.
- Windsurf: Este asistente se utilizó para autocompletar y/o predecir código, especialmente en ocasiones donde se puede considerar tediosa o repetitiva la escritura. Como ejemplo se puede tomar la creación de etiquetas e imágenes utilizando Tkinter, donde se debe repetir el mismo proceso una y otra vez, pero con diferentes nombres de variables y posiciones, donde este asistente detecta patrones y en base a ello genera sugerencias de cómo continuar el código, lo que resulta ser rápido y eficaz en un porcentaje de las veces. El código generado fue revisado de forma detallada para prevenir errores o problemas futuros en el funcionamiento del programa.

5.3. Declaración de Responsabilidad y Autoría

Como suscritos, integrantes del equipo de trabajo del proyecto “OvercookTEC”, declaramos que:

- El diseño, análisis, desarrollo y conclusiones del presente proyecto son resultado de trabajo intelectual propio y esfuerzo.
- Las herramientas de Inteligencia Artificial fueron utilizadas exclusivamente como apoyo en tareas específicas descritas en este capítulo, no como sustituto del trabajo.
- Todo el contenido generado por Inteligencia Artificial (IA) fue revisado, comprendido, validado y, cuando fue necesario, corregido, para cumplir con la calidad de la asignatura.
- Se asume plena responsabilidad académica sobre el contenido total de este proyecto y su documentación.
- El uso de IA se realizó conforme a las políticas de la institución educativa y los lineamientos del docente responsable.

Firman:



Ignacio José Apuy Anchía
Integrante del proyecto.



Jordanny Steven Hernández Mora
Integrante del proyecto.

7. Conclusión

La elaboración de este proyecto representó un reto significativo de cuatro semanas, el cual finalizó debidamente con las últimas revisiones al código fuente y al comportamiento del sistema para verificar que todo estuviese en orden. Finalmente se logró crear una réplica básica del videojuego original para dos jugadores, utilizando escenarios con casillas, donde los jugadores interactúan con cada celda según su función predefinida, y además con manejo de tiempos por partida, cálculo de puntajes, tiempos de cocción de los alimentos. Todo esto programado en lenguaje Python.

Se tomaron en cuenta la prueba de destrezas y el aprendizaje de nuevas técnicas y éticas, especialmente con el modelo de programación orientado a objetos, superando dificultades como el manejo adecuado de las instancias, o el modelo centralizado de controlador, así como sus vistas y las complejidades técnicas de sintaxis y funcionamiento de la librería Tkinter.

Como propuesta hacia futuras instancias de este proyecto, se pueden plantear otros lenguajes de programación con sintaxis más familiar a lo usual, debido a que Python, aunque es lo suficientemente potente y flexible, genera complejidad visual y problemas de manejo de tipos a la hora de programar. Además, se pueden utilizar librerías alternativas a Tkinter como lo son PyQt o Pygame para casos específicos como recrear un videojuego.

8. Bibliografía

- [1] Python Software Foundation, *Python 3.14 Documentation*, Python.org, 2026. [En línea]. Disponible en: <https://docs.python.org/3/>
- [2] Python Software Foundation, *Graphical User Interfaces with Tk (Tkinter)*, Python.org, 2026. [En línea]. Disponible en: <https://docs.python.org/3/library/tk.html>
- [3] International Organization for Standardization, *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*, ISO/IEC 25010:2011, Ginebra, Suiza, 2011.
- [4] Google LLC, *Site Reliability Engineering: How Google Runs Production Systems*, O'Reilly Media, 2016. [En línea]. Disponible en: <https://sre.google/sre-book/table-of-contents/>
- [5] Microsoft Corporation, *Software Testing Fundamentals*, Microsoft Learn, 2026. [En línea]. Disponible en: <https://learn.microsoft.com/en-us/shows/software-testing-fundamentals/>
- [6] Team17 / Ghost Town Games, *Recipes and Cooking Mechanics*, Overcooked! Wiki, 2026. [En línea]. Disponible en: https://overcooked.fandom.com/wiki/Overcooked_Wiki
- [7] Microsoft Corporation, *Visual Studio Code Documentation*, Microsoft, 2026. [En línea]. Disponible en: <https://code.visualstudio.com/docs>
- [8] GitHub, Inc., *GitHub Documentation*, GitHub Docs, 2026. [En línea]. Disponible en: <https://docs.github.com/>
- [9] Google LLC, *Google Developer Documentation Style Guide*, Google Developers, 2024. [En línea]. Disponible en: <https://developers.google.com/style>